BSc (Hons) in **Information Technology Specialized in AI**

Faculty of Computing

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Practical 04

> **Objective & Lecture Mapping:** In previous labs, you explored Uninformed Search strategies like BFS and UCS inside `agent.py` [cite: 1]. Today, we transition to **Informed Search**. You will enhance your agent by implementing the **A* Search** algorithm, using **Heuristics** to drastically reduce the number of explored nodes in the `visual_grid_game.py` environment[cite: 4]. You will start with basic heuristic functions and connect them to your search logic.

Part 1: Practical Implementation — Building the A* Agent

Step 1.1: Implementing the Heuristic Functions

> Informed search algorithms require a heuristic function, h(n), which estimates the cheapest path from the current node to the goal. You will calculate distance metrics on a 2D grid.

1. Create a new Branch called Week_04 in your Forked Github Repo.
2. Open `agent.py` and locate your `SearchAgent` class[cite: 1].
3. Add a new method named `def manhattan_distance(self, pos, goal):` that calculates the distance using the formula h(n) = |x_1 - x_2| + |y_1 - y_2| and returns the integer value.
4. Add a second method named `def euclidean_distance(self, pos, goal):` that calculates the straight-line distance using the formula h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}. You will need to `import math` for this.
5. *Testing Checkpoint:* Print the outputs of both functions for a mock start position (0, 0) and goal (3, 4) to verify that Manhattan returns 7 and Euclidean returns 5.0.

Step 1.2: Implementing A* Search

> A* evaluates nodes by combining the path cost so far, g(n), and the estimated cost to the goal, h(n). The evaluation function is f(n) = g(n) + h(n).

1. Inside `SearchAgent`, create a new method: `def astar_search(self, start_pos, goal_pos, walls, grid_size, heuristic_type='manhattan')`.
2. Initialize an empty priority queue using `heapq` and an empty set for `reached_states`.
3. Push the initial state into the priority queue. Unlike UCS which only uses g(n), your A* tuple must be formatted as: `(f_cost, g_cost, current_pos, path_taken)`. For the starting node, g(n) = 0. Calculate h(n) using your chosen heuristic method, and set f(n) = g(n) + h(n).
4. Create your standard `while` loop to process the queue. Pop the node with the lowest `f_cost`. If `current_pos == goal_pos`, return the `path_taken`. Otherwise, add `current_pos` to `reached_states`.
5. For node expansion, check the four adjacent cells (Up, Down, Left, Right). For each valid neighbor (not a wall, within bounds, and not reached): Calculate g_{new} = g_{current} + 1, h_{new} using your heuristic, and f_{new} = g_{new} + h_{new}. Push the new tuple to the priority queue.

Step 1.3: Integrating A* into the Agent's Decision Loop

> Finally, you need to connect your new search algorithm to the agent's perception and action loop so it can run in the visual environment.

1. Navigate to the `sense_and_act(self, percept)` method in your `SearchAgent`.
2. Add an `elif self.active_algo == 'AStar':` block to handle the new algorithm alongside your existing BFS/DFS/UCS.
3. Extract the necessary global state from the percept dictionary (e.g., `percept['remaining_food']`).
4. Find the closest food item to act as the `goal_pos`. Call your `astar_search` method and store the returned list of actions in `self.plan`.

5. Open `visual_grid_game.py` and modify the `GridGameGUI` initialization to inject your `SearchAgent()` instead of the `ModelBasedAgent()`. Run the simulation and observe the optimized pathfinding!

Part 2: Theoretical Evaluation

> Based on your implementation and Lecture 05, complete the following analytical questions.

1. (Understand) What is the key difference between how Uniform-Cost Search (UCS) and A* Search prioritize which node to explore next?

Both search algorithms use a priority queue, but they order it by different values. UCS prioritizes purely by g(n), the actual cost accumulated from the start node, so it always expands whichever node is cheapest to reach so far and pays no attention to where the goal is. A* prioritizes by f(n) = g(n) + h(n), adding a heuristic estimate of the remaining cost to the goal. This forward-looking term pulls the search toward the goal, so A* explores far fewer nodes than UCS while still returning an optimal path when the heuristic is admissible. In effect, UCS is just A* with h(n) set to zero.

2. (Analyze) In Step 1.1, you used Manhattan Distance. Why is Manhattan Distance considered an "admissible" heuristic for this specific 4-way movement grid, and what would happen to your A* algorithm if the heuristic was NOT admissible?

On this grid the agent moves only Up, Down, Left or Right, each step costs 1, and diagonal moves are impossible, so travelling between two cells requires at least $|X1-X2|$ horizontal steps plus $|Y1-Y2|$ vertical steps. That sum is exactly the Manhattan distance, and walls can only lengthen the real path, never shorten it, so Manhattan distance never overestimates the true cost and is therefore admissible. If the heuristic were not admissible and overestimated for some node, A* could inflate that node's f-value, settle on a sub-optimal route to the goal first, and return it. The algorithm would still find a path, but it would lose its guarantee of finding the shortest one.

3. (Evaluate) If we modified `visual_grid_game.py` to allow the agent to move diagonally (8-way movement), would Manhattan distance still be an admissible heuristic? Why or why not? Which metric should you switch to?

No. Once diagonal movement is allowed and a diagonal step also costs 1, a single move can reduce both the x and y gap at the same time, so the true cost from (0,0) to (3,3) is 3 moves while Manhattan distance reports 6. Because it now overestimates, Manhattan distance becomes inadmissible and A* may return a non-optimal path. The correct choice is Chebyshev distance, $\max(|X1-X2|, |Y1-Y2|)$, which is the exact minimum number of 8-way steps; if diagonals instead cost $\sqrt{2}$ you would use the octile distance. Euclidean distance also stays admissible but is looser, so A* would expand more nodes.

4. (Create) When targeting multiple food items simultaneously, calculating the distance to just the single closest food item is a weak heuristic. Propose (in text) a stronger heuristic for navigating the grid to eat ALL remaining food efficiently.

Measuring only the distance to the closest food ignores all the travel still needed to collect the rest, so it badly underestimates the remaining work. A stronger heuristic adds two admissible lower bounds: the Manhattan distance from the agent to the nearest remaining food, plus the total weight of a minimum spanning tree connecting all remaining food cells using Manhattan distances as edge weights. Any route that eats every item must connect them all, and the MST is the cheapest structure that does so, so the sum never overestimates and A* stays optimal. A cheaper approximation is "distance to the nearest food plus distance from that food to the farthest food," and if optimality is not required, a greedy nearest-neighbour tour length through all remaining food gives even sharper guidance at the cost of admissibility.

Once Completed Get a PDF of the completed File and Push into the Week 04 branch in the Github Project

Finalize and Lock Lab 04 Documentation

FACULTY OF COMPUTING